Desenvolvimento Web com .NET

Aula 8 – API Biblioteca
Listar Dados

Prof. Me. Lucas Teodoro dos Santos

UNIP

14 de Outubro de 2026

Roteiro da Aula

1. Criação da API Biblioteca
2. Swagger
3. Comunicação com o Banco de Dados
4. Método de Consulta dos Livros
5. Consulta Assíncrona

UNIP
UNIVERSIDADE PAULISTA

Criação do projeto Biblioteca

No Visual Studio, a criação do projeto utiliza as seguintes opções:

1. Seleção do modelo **API Web do ASP.NET Core**;
2. Nome do projeto: **Biblioteca**;
3. Framework: **.NET 9**;
4. Autenticação: **Nenhuma**;
5. Opção **Usar controllers** habilitada;
6. Opção **Configurar para HTTPS** habilitada.

Para esta implementação, a opção de suporte ao OpenAPI do template pode ser desmarcada, pois a documentação será configurada pelo Swashbuckle.

UNIP
UNIVERSIDADE PAULISTA

O que é Swagger?

Swagger é um conjunto de ferramentas para documentar e explorar APIs descritas pelo padrão **OpenAPI**.

A ferramenta **Swagger UI** apresenta uma interface no navegador que permite:

- Visualizar os endpoints disponíveis;
- Consultar os parâmetros e os formatos dos dados;
- Enviar requisições para a API;
- Inspecionar o código de status e a resposta recebida.

UNIP
UNIVERSIDADE PAULISTA

Instalação do Swagger

O pacote `Swashbuckle.AspNetCore` fornece os recursos para gerar a documentação e disponibilizar o Swagger UI.

No Visual Studio, o pacote pode ser instalado pelo **Console do Gerenciador de Pacotes**:

```
Install-Package Swashbuckle.AspNetCore
```

Outra opção é acessar:

Projeto → Gerenciar Pacotes NuGet

Na aba de pesquisa, a instalação utiliza o pacote **`Swashbuckle.AspNetCore`**.

Configuração do Program.cs

O conteúdo do `Program.cs` fica definido da seguinte forma:

```
var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllers();
builder.Services.AddSwaggerGen(); //Adicionar

var app = builder.Build();

//Adicionar trecho de codigo
if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}

app.UseHttpsRedirection();
app.UseAuthorization();
app.MapControllers();

app.Run();
```

Execução da API inicial

A aplicação deve ser iniciada no modo de depuração, pelo botão **Iniciar Depuração** do Visual Studio

O Swagger UI fica disponível no endereço da aplicação, acrescentando o caminho `/swagger`.

Exemplo de endereço

```
https://localhost:7001/swagger
```

A porta depende da configuração do projeto.

Caso o template tenha criado o endpoint `WeatherForecast`, ele aparece na interface e pode ser testado pelos botões **Try it out** e **Execute**.

UNIP
UNIVERSIDADE PAULISTA

Inicialização Automática do Swagger

Configure o perfil de execução no arquivo `Properties/launchSettings.json` para abrir o Swagger automaticamente ao iniciar a API.

```
{
    "$schema": "https://json.schemastore.org/launchsettings.json",
    "profiles": {
        "https": {
            "commandName": "Project",
            "dotnetRunMessages": true,
            "launchBrowser": true,
            "launchUrl": "swagger",
            "applicationUrl": "https://localhost:7189;http://localhost:5067",
            "environmentVariables": {
                "ASPNETCORE_ENVIRONMENT": "Development"
            }
        }
    }
}
```

UNIVERSIDADE PAULISTA

Projeto Biblioteca

A API Biblioteca permitirá gerenciar os livros de uma biblioteca e trabalhar com os seguintes recursos técnicos:

Operações

- Cadastrar livros;
- Consultar livros;
- Atualizar dados;
- Remover livros.

Tópicos Técnicos

- Entity Framework Core assíncrono;
- Data Annotations;
- Autenticação e autorização com JWT;
- Paginação, filtros e ordenação.

UNIP
UNIVERSIDADE PAULISTA

Entidade Livro

Na raiz do projeto, crie a pasta `Models`. Dentro dela, crie o arquivo `Livro.cs`.

```
using System.ComponentModel.DataAnnotations;

namespace APIBiblioteca.Models;

public class Livro
{
    public int Id { get; set; }

    [Required]
    [StringLength(150)]
    public string Titulo { get; set; } = string.Empty;

    [Required]
    [StringLength(100)]
    public string Autor { get; set; } = string.Empty;

    public bool Emprestado { get; set; }
}
```

Propriedades e Data Annotations da Entidade Livro

- `Id`: Identifica o livro e é reconhecido pelo Entity Framework Core como chave primária por convenção;
- `[Required]`: Indica que o título e o autor devem ser informados;
- `[StringLength(150)]` e `[StringLength(100)]`: Definem o limite de caracteres do título e do autor;
- `Emprestado`: Indica se o livro está emprestado. Como é do tipo `bool`, seu valor inicial é `false`.

As Data Annotations definem regras de validação e podem orientar a configuração do banco de dados.

UNIP
UNIVERSIDADE PAULISTA

Comunicação com o Banco de Dados - Instalação dos Pacotes

Para conectar a API ao PostgreSQL utilizando o Entity Framework Core, é necessário instalar os pacotes correspondentes. Escolha uma das opções:

- **Opção 1 – Gerenciador de pacotes NuGet:**
  - `Npgsql.EntityFrameworkCore.PostgreSQL`
  - `Microsoft.EntityFrameworkCore.Design`
  - `Microsoft.EntityFrameworkCore.Tools`
- **Opção 2 – Console do Gerenciador de Pacotes:**

```
1  Install-Package Npgsql.EntityFrameworkCore.PostgreSQL
2  Install-Package Microsoft.EntityFrameworkCore.Design
3  Install-Package Microsoft.EntityFrameworkCore.Tools
```

Versão do .Net

Use versões compatíveis com o projeto.

UNIVERSIDADE PAULISTA

Comunicação com o Banco de Dados – String de conexão

No arquivo `appsettings.json`, localizado na raiz do projeto, adicionar a seção `ConnectionStrings` e a string de conexão:

```
{
    "ConnectionStrings": {
        "DefaultConnection": "Host=localhost;Port=5432;Database=Biblioteca;Username=
            postgres;Password=admin"
    },
    "Logging": {
        "LogLevel": {
            "Default": "Information",
            "Microsoft.AspNetCore": "Warning"
        }
    },
    "AllowedHosts": "*"
}
```

Comunicação com o Banco de Dados - Classe de Contexto

Após instalar os pacotes, crie a pasta `Data` na raiz do projeto. Nela, adicione o arquivo `ApiDbContext.cs` para definir o contexto do banco de dados.

```
using APIBiblioteca.Models;
using Microsoft.EntityFrameworkCore;

namespace APIBiblioteca.Data;

public class ApiDbContext : DbContext
{
    public ApiDbContext(DbContextOptions<ApiDbContext> options)
    : base(options)
    {
    }

    public DbSet<Livro> Livros => Set<Livro>();
}
```

Comunicação com o Banco de Dados – Registrando o Serviço

Para que a API acesse o PostgreSQL por meio do Entity Framework Core, o `ApiDbContext` deve ser registrado como um serviço da aplicação. Essa configuração é feita no arquivo `Program.cs`, antes de `builder.Build()`.

```
using APIBiblioteca.Data;
using Microsoft.EntityFrameworkCore;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddDbContext<ApiDbContext>(options =>options.UseNpgsql(
    builder.Configuration.GetConnectionString("DefaultConnection")));
```

Comunicação com o Banco de Dados – Primeira Migration

Com a entidade e o contexto criados, execute os comandos no **Console do Gerenciador de Pacotes**:

```
1  Add-Migration PrimeiraMigration
2  Update-Database
```

- `Add-Migration` cria a migration inicial.
- `Update-Database` aplica as alterações ao banco de dados.

UNIP
UNIVERSIDADE PAULISTA

Seeding com HasData

O seeding com `HasData` define dados iniciais que serão gerenciados pelo modelo do Entity Framework Core.

- A configuração é feita no método `OnModelCreating`, na classe que herda de `DbContext`;
- Neste projeto, o método será implementado na classe `ApiDbContext`;
- O EF Core utiliza os dados configurados para gerar operações de inserção nas migrations;
- Cada registro precisa ter um valor definido para a chave primária.

No arquivo `Data/ApiDbContext.cs`, sobrescreva `OnModelCreating` de acordo com o próximo slide.

UNIP
UNIVERSIDADE PAULISTA

Sobrescrevendo OnModelCreating

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    base.OnModelCreating(modelBuilder);

    Livro l1 = new()
    {
        Id = 1,
        Titulo = "Dom Casmurro",
        Autor = "Machado de Assis",
        Emprestado = false
    };

    Livro l2 = new()
    {
        Id = 2,
        Titulo = "O Pequeno Principe",
        Autor = "Antoine de Saint-Exupery",
        Emprestado = false
    };

    modelBuilder.Entity<Livro>().HasData(l1, l2);
}
```

Aplicando o Seeding

Após configurar os dados com `HasData`, crie uma nova migration para registrar os livros iniciais.

No **Console do Gerenciador de Pacotes**, execute:

```
1           Add-Migration InserirLivrosIniciais
2           Update-Database
```

LivrosController

A controller organiza os endpoints da API e recebe as requisições HTTP relacionadas a um recurso.

Na API Biblioteca, a classe `LivrosController` será responsável pelas operações relacionadas aos livros:

- Receber as requisições do cliente;
- Retornar uma resposta HTTP com o resultado da operação.

Cada método da controller poderá corresponder a um endpoint da API.

UNIP
UNIVERSIDADE PAULISTA

Estrutura Inicial da LivrosController

Crie o arquivo `Controllers/LivrosController.cs`:

```
using Microsoft.AspNetCore.Mvc;

namespace APIBiblioteca.Controllers;

[ApiController]
[Route("api/[controller]")]
public class LivrosController : ControllerBase
{
}
```

Rotas da Controller

A rota definida na classe funciona como prefixo para os endpoints.

```
[Route("api/[controller]")]
public class LivrosController : ControllerBase
{
    [HttpGet]
    // GET /api/livros

    [HttpGet("{id:int}")]
    // GET /api/livros/5

    [HttpGet("buscar")]
    // GET /api/livros/buscar
}
```

- `[controller]` é substituído pelo nome da classe sem o sufixo `Controller`;
- O atributo HTTP da ação, como `[HttpGet]`, complementa a rota da classe;
- `{id:int}` define um parâmetro de rota chamado `id` e restringe seu valor a um número inteiro.

LivroService

A `LivroService` é uma classe da aplicação criada para concentrar as operações relacionadas aos livros.

Ela faz a ligação entre a controller e o acesso aos dados:

- A controller recebe a requisição HTTP e chama um método do serviço;
- A `LivroService` executa a operação solicitada;
- O serviço utiliza o `ApiDbContext` para consultar ou alterar os dados;
- O Entity Framework Core traduz essas operações para comandos enviados ao PostgreSQL.

Assim, a controller não precisa conhecer os detalhes da consulta ao banco.

Responsabilidades da LivroService

A `LivroService` não é uma classe pronta do ASP.NET Core. Ela é criada para organizar as operações específicas da aplicação.

Na API Biblioteca, esse serviço poderá concentrar métodos para:

- Listar os livros;
- Localizar um livro pelo identificador;
- Cadastrar um livro;
- Atualizar os dados de um livro;
- Remover um livro.

A controller ficará responsável pela comunicação HTTP, enquanto o serviço organizará as operações sobre os dados dos livros.

UNIP
UNIVERSIDADE PAULISTA

Criando a Classe LivroService

Crie a pasta `Services` e, dentro dela, o arquivo `LivroService.cs`. A classe concentrará as operações relacionadas aos livros.

```
using APIBiblioteca.Data;
using APIBiblioteca.Models;

namespace APIBiblioteca.Services;

public class LivroService
{
    private readonly ApiDbContext _context;

    public LivroService(ApiDbContext context)
    {
        _context = context;
    }
}
```

LivroService – Método de Consulta

Na classe `LivroService`, implemente o método `Listar` para consultar os livros cadastrados.

Neste primeiro momento, a consulta ao banco será síncrona:

```
1           public List<Livro> Listar()
2           {
3               return _context.Livros.ToList();
4           }
```

Registrando o Serviço – LivroService

Para que o ASP.NET Core possa fornecer a `LivroService` à controller, registre o serviço no arquivo `Program.cs`, antes de `builder.Build()`:

```
1           using APIBiblioteca.Services;
2
3           builder.Services.AddScoped<LivroService>();
```

Com o ciclo de vida `Scoped`, uma instância do serviço é usada durante cada requisição HTTP.

Método de Consulta dos Livros

A controller recebe a `LivroService` por injeção de dependência. Assim, pode solicitar as operações disponíveis na classe de serviço.

Na classe `LivrosController`, adicione o campo e o construtor:

```
1           private readonly LivroService _livroService;
2
3           public LivrosController(LivroService livroService)
4           {
5               _livroService = livroService;
6           }
```

Implementando a consulta na LivrosController

Após o construtor, adicione o método que responde à requisição `GET /api/livros`:

```
1           [HttpGet]
2           public ActionResult<List<Livro>> Listar()
3           {
4               var livros = _livroService.Listar();
5               return Ok(livros);
6           }
```

- `[HttpGet]` associa o método a uma requisição HTTP GET;
- `_livroService.Listar()` solicita a consulta ao serviço;
- `Ok(livros)` retorna `200 OK` com a lista serializada no corpo da resposta.

Testando a Consulta no Swagger

Com a API em execução, teste a consulta dos livros pelo Swagger:

1. Acesse o endpoint `GET /api/livros`;
2. Clique em **Try it out** e, em seguida, em **Execute**;
3. Verifique a resposta `200 OK` e a lista de livros retornada em JSON.

Os livros inseridos pelo seeding, como *Dom Casmurro* e *O Pequeno Príncipe*, devem aparecer na resposta.

UNIP
UNIVERSIDADE PAULISTA

Consulta Assíncrona

Na implementação atual, a requisição aguarda a resposta do banco enquanto a thread permanece ocupada. Com várias consultas simultâneas, isso pode reduzir a capacidade da API de atender outras requisições.

Para melhorar o uso desses recursos, a consulta será refatorada para a programação assíncrona, permitindo aguardar o banco sem bloquear a thread durante esse período.

Atualmente, o método da `LivroService` é síncrono:

```
1           public List<Livro> Listar()
2           {
3               return _context.Livros.ToList();
4           }
```

Consulta Assíncrona na LivroService

A consulta assíncrona permite aguardar o banco sem manter a thread ocupada durante a resposta.

Na `LivroService`, substitua o método síncrono por:

```
1           public async Task<List<Livro>> ListarAsync()
2           {
3               return await _context.Livros.ToListAsync();
4           }
```

- `async`: Permite aguardar uma operação assíncrona;
- `Task<List<Livro>>`: Indica que o método fornecerá uma lista de livros quando a operação terminar;
- `ToListAsync()`: Versão assíncrona de `ToList()`, que executa a consulta e monta a lista;
- `await`: Aguarda o resultado antes de retorná-lo, liberando a thread durante a espera.

Consulta Assíncrona na LivrosController

Como a consulta da `LivroService` agora é assíncrona, a `Controller` também precisa aguardar sua conclusão antes de montar a resposta HTTP. Assim, a chamada assíncrona se mantém desde o acesso ao banco até o atendimento da requisição.

Atualize o método de consulta:

```
[HttpGet]
public async Task<ActionResult<List<Livro>>> Listar()
{
    var livros = await _livroService.ListarAsync();
    return Ok(livros);
}
```

- `async` permite aguardar a operação da service;
- `Task<ActionResult<List<Livro>>>` indica que o método retornará uma resposta HTTP com uma lista de livros;
- `await` aguarda a lista antes de continuar a execução;
- `Ok(livros)` mantém a resposta `200 OK` com a lista no corpo da resposta.

Fluxo Após a Refatoração

A operação assíncrona percorre as camadas da aplicação:

1. A Controller recebe a requisição `GET /api/livros`;
2. A Controller aguarda a chamada à `LivroService`;
3. A Service aguarda a consulta assíncrona do EF Core;
4. Quando o banco retorna os dados, a lista volta à Controller;
5. A Controller responde com `200 OK` e os livros em JSON.

A rota e o formato da resposta permanecem iguais. A mudança está na forma como a aplicação aguarda a consulta ao banco.

UNIP
UNIVERSIDADE PAULISTA

Tratamento de Exceções

Uma das formas de tratar falhas em uma API é permitir que a `Service` sinalize o problema por meio de uma exceção e que a `Controller` a capture para gerar uma resposta HTTP adequada.

Nessa abordagem, a `Service` concentra a operação sobre os dados, enquanto a `Controller` traduz a falha para o formato da API. Existem outras estratégias, como o tratamento global de exceções.

Na consulta atual, uma lista vazia é um resultado válido. A exceção será considerada quando ocorrer uma falha durante a consulta ao banco.

UNIP
UNIVERSIDADE PAULISTA

Lançando uma Exceção na LivroService

A consulta continua sendo feita pela `LivroService`. Se ocorrer uma falha, o serviço lança uma exceção com uma mensagem apropriada:

```
1           public async Task<List<Livro>> ListarAsync()
2           {
3               try
4               {
5                   return await _context.Livros.ToListAsync();
6               }
7               catch (Exception ex)
8               {
9                   throw new InvalidOperationException(
10                  "Nao foi possivel consultar os livros.", ex);
11              }
12          }
```

A exceção original é mantida como causa interna. Assim, a falha pode ser investigada sem expor seus detalhes diretamente ao cliente da API.

Capturando a Exceção na LivrosController

A `Controller` captura a exceção e retorna uma resposta HTTP que informa ao cliente que ocorreu uma falha no servidor:

```
[HttpGet]
public async Task<ActionResult<List<Livro>>> Listar()
{
    try
    {
        var livros = await _livroService.ListarAsync();
        return Ok(livros);
    }
    catch (InvalidOperationException)
    {
        return StatusCode(500, new
        {
            mensagem = "Nao foi possivel consultar os livros."
        });
    }
}
```

Se a consulta for concluída, a API retorna 200 `OK`. Se ocorrer a falha tratada, retorna 500

Inserção de livros

A funcionalidade permitirá enviar os dados de um novo livro à API. A controller receberá as informações da requisição e solicitará à `LivroService` que grave o registro no banco de dados.

As regras definidas com *Data Annotations* serão aplicadas aos dados recebidos. Após a gravação, a API retornará o livro cadastrado.

UNIP
UNIVERSIDADE PAULISTA

Método Inserir – LivroService

No arquivo `Services/LivroService.cs`, adicione o método `InserirAsync` após o método `ListarAsync`:

```
1           public async Task<Livro> InserirAsync(Livro livro)
2           {
3               _context.Livros.Add(livro);
4
5               try
6               {
7                   await _context.SaveChangesAsync();
8                   return livro;
9               }
10              catch (DbUpdateException ex)
11              {
12                  throw new InvalidOperationException(
13                  "Nao foi possivel cadastrar o livro.", ex);
14              }
15          }
```

`Add` prepara a entidade para inserção. A gravação acontece em `SaveChangesAsync`.

Método InserirAsync

O método adiciona o livro ao contexto do Entity Framework Core e solicita a gravação assíncrona no banco.

- `Add` prepara a entidade para ser inserida;
- `SaveChangesAsync` grava os dados e atualiza o `Id` gerado pelo banco;
- Se o EF Core lançar uma `DbUpdateException`, o método a encapsula em uma `InvalidOperationException`, mantendo a exceção original para investigação.

Endpoint de Inserção – LivroController

No arquivo `Controllers/LivrosController.cs`, adicione este método após a ação `Listar`:

```
[HttpPost]
public async Task<ActionResult<Livro>> Inserir(
[FromBody] Livro livro)
{
    try
    {
        var livroCriado =
        await _livroService.InserirAsync(livro);

        return StatusCode(201, livroCriado);
    }
    catch (InvalidOperationException ex)
    {
        return StatusCode(500, new
        {
            mensagem = ex.Message
        });
    }
}
```

Endpoint de Inserção

O endpoint recebe os dados do livro, solicita a inserção à `LivroService` e transforma o resultado em uma resposta HTTP.

- `[HttpPost]` associa o método a uma requisição POST;
- `[FromBody]` obtém o livro enviado no corpo da requisição;
- `await` aguarda a gravação antes de retornar `201 Created`;
- Se a service lançar uma exceção, o método retorna `500 Internal Server Error`.

Com `[ApiController]`, dados inválidos são rejeitados automaticamente com `400 Bad Request`.

Testando o endpoint no Swagger

No Swagger, abra o endpoint `POST /api/livros`, selecione **Try it out** e envie um corpo como este:

```
{
    "titulo": "Quincas Borba",
    "autor": "Machado de Assis",
    "emprestado": false
}
```

- O campo `Id` pode ser omitido;
- Com os dados válidos e a gravação concluída, a resposta será `201 Created`;
- Se ocorrer uma falha durante a gravação, a controller retorna `500 Internal Server Error`.

Resumo Geral da Aula

- Criação da API Biblioteca e configuração do Swagger para documentar e testar os endpoints;
- Modelagem da entidade `Livro` com propriedades e *Data Annotations*;
- Configuração do PostgreSQL com Entity Framework Core, migrations e carga inicial de dados com `HasData`;
- Organização das responsabilidades entre `LivrosController` e `LivroService`, com a implementação da consulta dos livros;
- Implementação da consulta dos livros;
- Refatoração da consulta para o modelo assíncrono, utilizando `async`, `await` e `ToListAsync()`.
- Tratamento de exceções na service e conversão das falhas em respostas HTTP pela controller.
- Implementação da inserção de livro;

UNIP
UNIVERSIDADE PAULISTA

Obrigado!

Dúvidas?

UNIP
UNIVERSIDADE PAULISTA